

李乐岩

leyanpower@gmail.com | +86 18805506898 | GitHub: github.com/leyancode | 个人主页: leyan-li.com

爱丁堡大学 University of Edinburgh 英国 | 2025.09 – 2026.12 高性能计算与数据科学 硕士 课程与毕业论文 2026.08 完成 | 可入职时间 2026.09.01 | 学位证书 2026.12 获得

利物浦大学 University of Liverpool 英国 | 2023.09 – 2025.06 计算机科学 本科, 西交利物浦大学 2+2 项目, 一等荣誉学位

技术能力

并行与分布式性能工程: MPI、OpenMP、hybrid MPI+OpenMP、PETSc、Slurm; 在英国国家超算 ARCHER2 上完成最多 32 节点 / 4,096 核的正式对照实验。理解 strong / weak scaling、parallel efficiency 与饱和点判定, 以及 halo exchange、CG global reduction (allreduce) 等集合通信开销与 surface-to-volume ratio 对通信暴露的影响。

性能剖析与体系结构: Linaro MAP call-tree profiling; 可从 NUMA / CCX / 共享 L3、memory bandwidth、线程绑定 (OMP_PLACES、OMP_PROC_BIND) 与负载均衡角度解释瓶颈, 并把开销拆成计算、通信、同步等待三类分别归因。

数值方法与求解器: PETSc KSP / PC 栈: CG + GAMG 代数多重网格、GMRES + block Jacobi/ICC, 理解 preconditioner 随分区变化对可比性的破坏; 2D five-point / 3D seven-point stencil 离散与已知解误差校验; 理解迭代数、每迭代成本与总吞吐三者需要分开看。

编程与工程: C/C++、Python、Bash、SQL; Linux、Git、pytest、GitHub Actions CI, 了解 Docker。习惯把实验做成可复现管线: 参数化提交脚本 → 基于内容的日志审计 → pandas 聚合 → 图表与回归测试。

数据分析与可视化: pandas、NumPy、Matplotlib; 实验日志解析、指标计算、中位数与观测区间的误差表达, 以及把关键数值锁进测试以防图表与结论漂移。

项目经历: ARCHER2 上 PETSc hybrid MPI+OpenMP 布局的可审计性能实验 (32 节点 / 4,096 核)
github.com/leyancode/hpc_benchmark_archer2

项目描述: 在英国国家超算 ARCHER2 (HPE Cray EX, 双路 AMD EPYC 7742、128 核/节点, Slingshot 互连) 上研究一个受约束的配置问题: 总核数固定时, 减少 MPI rank 数、增大每 rank 的 OpenMP 线程数, 能否用省下的通信与进程开销抵消线程组创建、barrier 与空转成本。负载为 2D five-point / 3D seven-point stencil + CG+GAMG, 规模 5M-165M unknowns, 比较四种满节点布局 (128×1 / 64×2 / 32×4 / 16×8)。正式结论只使用 408 个通过内容审计的 run (264 weak-scaling + 144 fixed-size strong-scaling)。

技术栈: C、PETSc、MPI、OpenMP、Slurm、Bash、Python、pandas、Matplotlib、Linaro MAP、GitHub Actions、Linux

- 实验设计:** 2D/3D 按 unknown 数对齐而非按网格边长, 使同一线性系统规模下可比较两种 stencil 的通信暴露; weak scaling 以 $25,000 \leq \text{unknowns/core} \leq 100,000$ 的准入规则组织出 ~40k (6 点) 与 ~80k (5 点) 两条固定本地工作量的链, fixed-20M 则有意越过该窗口降到 ~4.9k unknowns/core 以观察布局何时饱和。计时只读 PETSc RepeatedSolves stage (1 次 warm-up 建 GAMG 层次、19 次复用 preconditioner), 不把 setup 与稳态求解混为一谈; 每个正式配置 3 次重复, 取中位数、误差线为观测 min-max。
- 主动作废不可信结果:** 审计发现 launcher 时代的日志有 302 / 303 份的实际 MPI process 数与文件名不符 (一份文件名请求 2048 ranks, PETSc header 只报 32), 据此废弃该阶段全部性能曲线, 并把「从日志内容而非文件名解析 process / thread 数」写成分析入口的硬校验; 另发现 default GMRES + block Jacobi/ICC 会随 rank 数改变 preconditioner、一次 --ksp_rtol override 被 PETSc 静默忽略, 因此禁止用该批数据做跨 rank 布局排名。
- 可审计聚合管线:** 每份日志须通过 PETSc 自报 architecture、实际 ranks × threads、20 次 solve 全部收敛、timed stage 恰好 19 个 KSPSolve 且无 PCSetUp、链接库与 solver signature 共六项检查才进入聚合; 未通过的标为 superseded, 只解释方法如何改进, 永不混进中位数。
- Weak-scaling 结论:** 2D 上 64×2 在 11 个 size/node 点中赢 8 个, 165M / 32 节点达 794.8M equations/s, 比 flat-MPI 128×1 高 13.1%; 3D 上 32×4 赢 6 个, 最大吞吐增益 33.2% (41M / 4 节点, 对应 24.9% 时间节省)。结论是最佳布局随维度与规模改变, 不存在单一「最优线程数」; 由于 GAMG 迭代数会随分区小幅变化, 同时比较吞吐量与 seconds per KSP iteration。
- Strong-scaling 结论:** 固定 20M、1→32 节点: 2D 最快为 64×2, 0.0277 s/solve、29.83× 加速、93.2% 累积效率; 3D 最快为 16×8, 0.0681 s/solve、14.24× 加速、比 flat MPI 快 3.81×。各布局饱和点不同 (128×1 在 8 节点后变慢, 64×2 与 32×4 在 16 节点后变慢, 16×8 到 32 节点仍在下降) ——即每核工作量下降到一定程度后, 更粗的进程粒度才开始有回报; 这与固定 batch 继续扩机器时的收益递减是同一类问题。
- Profiling 定位机理:** Linaro MAP call tree 显示 threads/rank 上升时, solve window 内 MPI 占比从 46.7% (64×2) 降到 6.2% (16×8), 而 OpenMP runtime 等待从 48.3% 升到 76.5%——布局排序即「通信暴露」与「线程空转」两条曲线的交点; 同时排除 threaded BLAS 的解释 (LibSci 最多只占 solve-window core-time 的 2.1%)。因 MAP 插桩会不均匀扰动运行时间, 只读布局之间的占比排序, 绝不与未插桩的 scaling timing 混用。
- 工程化与可发布性:** Slurm 提交脚本参数化生成全部正式配置, 分析侧 16 个 Python 脚本配 pytest 断言公开矩阵行数、头条数值、README 图片与跨维度换算, GitHub Actions 跑公开数据测试并用 fake Slurm 执行提交脚本; 对外发布的 5 份代表日志经脚本脱敏 (去用户、allocation、node name、job ID 与个人路径), 并附私有原件与公开副本的 SHA-256 manifest。
- 局限如实标注:** 3 次重复的误差线是观测 min-max 而非置信区间; 结论绑定给定 stencil / solver / PETSc build / placement policy, 不代表所有 PETSc 应用都有相同最佳布局; 样本中可重复观察到的 superlinear speedup 未能确定硬件原因, 写为未解释项而非卖点。

面向该 benchmark 数据的 Agent 式性能分析系统 github.com/leyancode/hpc-benchmark-agent

项目描述：把上一项目的 PETSc benchmark 数据结构化为 SQLite，将性能分析任务封装为确定性工具，以中文规则 router 作为可验证 baseline，再接入 LLM function calling router，用自建 30 问五类中文评测集给两者打同一份分——规则 83.3%、LLM 100%。命题是：让 LLM 只负责「选哪个工具、参数是什么」，数值一律由确定性代码算，从而让 Agent 的正确性变成可评测量。

技术栈：Python、SQLite、pytest、FastAPI、OpenAI SDK (function calling)、Pydantic、Git、Linux

- **分层架构与同构契约：**数据导入 / 指标 / 工具 / 检索 / 路由 / 可观测六层，加 CLI 与 FastAPI 两个平级且互不依赖的进程边界；规则 router 与 LLM tool call 输出严格同构 (intent + 参数)，因此可共用同一套评测集、切换只需改一行 dispatch。各层合同由 229 项自动化测试钉死，全量离线可跑、不需要 API key (LLM 部分走 FakeLLM 依赖注入)。
- **LLM function calling、防御式解析与评测：**tool schema 的 description 写明工具边界，unknowns 刻意可选——省略参数即「追问」、不调用工具即「拒答」；未调用工具、幻觉工具名、坏 JSON 三类异常统一收敛为显式 unknown，绝不猜参数以避免「静默算错」。自建 30 问五类中文评测集 (期望值是 tool call 而非答案文本) 给两个 router 打同一份分，消融显示同一模型去掉 system prompt 只有 63.3%——准确率主要由工具说明书与策略 prompt 决定。
- **工程化外围：**trace_id 贯穿的 JSON Lines 事件流可完整回放一次执行，脱敏收敛在单一出口点并由扫描测试强制；FastAPI 暴露 POST /ask，非法 router 由 Pydantic 拦为 422、缺 key 映射 503 而非 500，与 CLI 的退出码 2 各自独立；最小 RAG 用 IDF 加权词法检索 (纯标准库、刻意不用向量库)，检索不到即返回空、不注入，未开启时 messages 与评测时逐字节一致并由测试锁死，以保结论可比。

多智能体 Actor-Critic 方法中的价值函数分解研究 (本科毕业设计) github.com/leyancode/marl-value-factorisation

项目描述：在 MAPPO 的 critic 上分别接入 VDN 加性分解与 QMIX 单调 mixing 做单变量对照：三个变体共用同一套 runner / buffer / shared RNN actor 与全部 PPO 超参，差异只在 critic 分支。结论与直觉相反——表达能力更强的 critic 没有赢：稀疏奖励的 Predator-Prey (300k steps) 上，最简单的加性分解唯一学到正回报 (≈ 150)，单体 critic 始终为负 ($-300 \rightarrow -30$)，单调 mixing 的 critic loss 起手即 5.4×10^4 、200k 步后二次发散。技术栈：Python、PyTorch、PyMARL、MAPPO、VDN、QMIX、CTDE、PPO、PettingZoo / MPE2。

- **训练不收敛的根因诊断：**MAPPO-QMIX 的 TD 目标沿用 value-based QMIX 的 $\max_a Q_i$ (贪婪动作)，而 actor 是 on-policy，相减得到的 td_error 并非合法 advantage 而是系统性偏乐观的量，可解释「回报冲过理论最优值后崩塌」；另定位 hypernet 经 softplus 生成的 mixing 权重 ($\text{softplus}(0) \approx 0.69$) 对局部 Q 的梯度做乘性放大，是 critic loss 起手即 5×10^4 的直接原因。
- **负结果与工程化：**去掉单调约束的 non-monotonic mixer 持续梯度爆炸、未在预算内收敛，如实写进论文；并指出三个 learner 实际都只产生一个共享标量 advantage，收益来自表示与梯度路径而非真正的 per-agent credit assignment。2026 年整理为可复现仓库：修正 Predator-Prey 合作边界、用 uv.lock 锁 CPU 版 PyTorch 使实验无需 GPU、补 pytest 并接入 GitHub Actions CI。

论文发表：Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis. International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.